MBU DIGITAL LIBRARY · PRACTICE LEARNING RESOURCE

C++ Programming

CPP101 | Suggested sequence · Semester 2 | Student-authored study guide

How to use these notes

Modern C++ foundations with value semantics, standard containers, algorithms, and resource-safe code. Work through the units in order, reproduce each example, then answer the review prompts from memory. This is supplemental practice material, not an official MBU syllabus or assessment. Course sequencing is suggested and should be checked against your current approved syllabus.

MODULE 01 · Types, functions, and references

Use const to express read-only intent, references for aliases, and clear value-returning functions. Prefer initialization syntax and avoid uninitialized variables. C++ overloads functions by parameter types.

Worked example

```
int square(const int value) { return value * value; }
```

Check your understanding

- When is a const reference useful?
- How is a reference different from a pointer?

MODULE 02 · Containers and algorithms

Use std::vector for a resizable sequence, std::string for text, and std::map/unordered_map for key lookup. Standard algorithms such as sort and find communicate intent and reduce loop mistakes.

Worked example

```
std::vector<int> scores{82, 91, 75}; std::sort(scores.begin(), scores.end());
```

Check your understanding

- Why choose vector over a raw array?
- What precondition does binary_search require?

MODULE 03 · Classes and resource ownership

Encapsulate invariants behind a small public interface. RAII ties resource lifetime to object lifetime. Prefer standard-library ownership types and containers; avoid manual new/delete when a value or smart pointer expresses ownership better.

Worked example

```
class Counter { public: void increment() { ++value_; } int value() const { return value_; }
private: int value_ = 0; };
```

Check your understanding

- What does RAII mean?
- Why should a class preserve its invariants?

Revision checklist

- %¡ Can I explain the main idea without copying the notes?
- %¡ Can I reproduce the example and state its assumptions?
- %¡ Can I test an edge case and describe what should happen?
- %¡ Have I recorded a limitation and checked a primary reference?

Further reading

Use the official documentation and open references linked in the portal. Check the publication/version and license at the source before redistribution. Cite the source and access date in coursework.